



CSE406: Cyber Security Sessional

TCP Reset Attack on Video Streaming

Final Report — Project Tool #15

2105094: Omar Abdur Razzaque
2105104: Rubaiyat -E-Zaman

September 19, 2026

Abstract

We implement, from scratch, a TCP reset (RST) attack against a video streaming session: an on-path attacker forges a **RST** segment that appears to originate from the legitimate server, using a sequence number read directly off the victim's own outgoing traffic rather than guessed, terminating the client's connection and interrupting playback. We describe the attack's design and implementation, the expected and actually observed outcomes (including one case where they diverge, and why), a switch-side countermeasure (IP Source Guard) that defeats the attack, and quantitative, repeated-trial measurements of both the attack's success rate and the defense's effectiveness across a range of network conditions.

Contents

1	Introduction	1
2	Threat Model and Assumptions	2
3	Topology and Implementation	2
3.1	Network Topology	2
3.2	Streaming Server and Client	2
3.3	Gaining Attacker Visibility	3
3.4	Forging the Reset Packet	3
4	Attack Process and Expected Outcome	4
5	Observed Outcome	4
5.1	Qualitative Observations	4
5.2	Quantitative Results	6
5.3	Expected vs. Observed Outcome, and Where They Differ	7
6	Defense Mechanism	8
6.1	Design	8

6.2 Defense Evaluation: Does It Work as Expected?	8
7 Discussion, Scope, and Limitations	9
8 Conclusion	10

1 Introduction

This report covers the implementation and evaluation of course project Tool #15: a TCP reset attack on video streaming. An on-path attacker, sharing a switched LAN with a video-streaming client, forges a TCP RST segment that appears to come from the legitimate streaming server. The forged RST kills the client’s connection immediately, interrupting playback, without guessing any sequence number – the value used is read directly from the client’s own outgoing traffic.

The rest of this report is organized as follows. Section 2 states the threat model and the assumptions the attack depends on. Section 3 describes the emulated topology and the implementation of the streaming service and the attacker. Section 4 describes the attack process step by step and the expected outcome. Section 5 reports what was actually observed, both qualitatively (logs, screenshots) and quantitatively (repeated-trial success rates across network conditions), and discusses the one case where the observed outcome diverged from what was expected. Section 6 describes and evaluates a switch-side countermeasure. Section 7 discusses limitations and scope, and Section 8 concludes.

2 Threat Model and Assumptions

The attacker is *on-path*: it shares the same switched LAN segment as the video-streaming client, while the real streaming server is remote, reachable only through a router. The attacker does not suppress, delay, or otherwise interfere with legitimate traffic – it only observes the client’s traffic and injects forged packets alongside it. The target is the TCP layer, independent of the HTTP version in use.

Explicit assumption: on-path visibility. A switched LAN does not, by default, deliver one host’s unicast traffic to another host on the same switch. Real-world attackers typically obtain this visibility via ARP spoofing, which poisons the ARP caches of the client and its gateway so that frames addressed to each other are instead delivered to the attacker. **This project explicitly assumes that this precondition has already been satisfied** by the time the attack begins. ARP spoofing itself is a separately assigned project tool (Tool #1), and reimplementing it here would duplicate that tool’s scope rather than demonstrate the RST-specific mechanics this tool is graded on. In the emulated topology (Section 3), we reproduce ARP spoofing’s *data-plane consequence* – the attacker being able to see the client’s traffic – directly, via a switch port mirror, rather than by performing the spoofing itself.

Other assumptions. The server is assumed to be running an ordinary HTTP/1.1 streaming service (no custom protocol hardening beyond what a standard Linux TCP stack already provides, e.g. RFC 5961 protections, which are discussed explicitly in Section 5). Only *Attack A* – a forged RST delivered to the *client*, spoofed as the server – is implemented and evaluated; the converse (spoofing the client toward the server) is architecturally straightforward given the same sequence-number-inference trick, but is not required and not evaluated here.

3 Topology and Implementation

3.1 Network Topology

The entire topology is emulated in code using Mininet and Open vSwitch – no physical or virtual routers/switches, and no separate virtual machines. Figure 1 shows the layout.

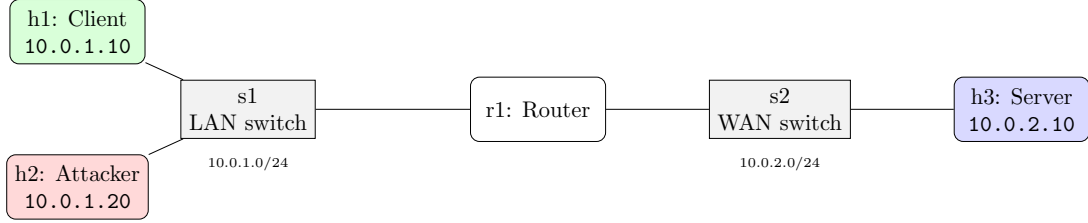


Figure 1: Emulated topology. h1 (client) and h2 (attacker) share switch s1 – one L2 hop apart. The server h3 is reachable only via router r1, a longer, bandwidth/delay-shaped path.

h1 and h2 sharing switch s1 gives the attacker its structural advantage: a forged packet from h2 to h1 takes one switched hop, while the server’s real response must cross the router-mediated, bandwidth/delay-shaped WAN link. Table 1 summarizes the technology used for each component.

Component	Technology
Network emulation	Mininet (Python API), Open vSwitch, <code>tc/netem</code>
Router	Plain Linux host, <code>ip_forward=1</code>
Streaming server	Flask (HTTP/1.1, Range-request / 206 support)
Client	<code>ffplay</code> (demo) / <code>curl</code> (scripted trials)
Attacker visibility	OVS port mirror (emulates ARP-spoof’s data-plane effect)
Attacker (sniff + craft)	Raw <code>AF_PACKET</code> sockets, hand-rolled header construction
Defense	OVS OpenFlow ACL rule (IP Source Guard)

Table 1: Implementation stack. All frame/packet/segment crafting is our own code; no third-party attack tool is used.

3.2 Streaming Server and Client

The server is a small Flask application serving a single video file with HTTP Range-request support (206 `Partial Content`), so a player can open it as a progressive-download stream rather than waiting for a full download. The test video is a synthetic clip generated with `ffmpeg` (a test pattern with a burnt-in running timestamp, so playback freezes are visually obvious), avoiding any copyrighted material.

3.3 Gaining Attacker Visibility

As stated in Section 2, we assume the ARP-spoofing precondition is already satisfied. To still demonstrate the attack against real traffic in the emulated topology, we reproduce ARP spoofing’s *data-plane consequence* directly: an Open vSwitch port mirror on switch s1 copies every frame entering or leaving h1’s switch port onto a second, dedicated monitor interface on h2. h2’s monitor interface is put into promiscuous mode, since the mirrored frames retain their original destination MAC address rather than being re-addressed to h2.

3.4 Forging the Reset Packet

The central design insight, on which the whole attack depends, is:

$$\text{client's outgoing ACK field} = \text{server's next-expected sequence number (server RCV.NXT)} \quad (1)$$

Because of this, the attacker never needs to observe a server-to-client packet at all. Sniffing a client-to-server segment (which, via the mirror, travels the attacker's own short path) is sufficient to read off the exact sequence number a real RST from the server would need. Per RFC 5961, an *exact* sequence match on a RST is accepted and the connection reset immediately; only an in-window-but-inexact value is downgraded to a challenge ACK, and an out-of-window value is silently discarded. Because our value is sniffed rather than guessed, it always satisfies the exact-match case – RFC 5961's headline protection, aimed at blind/guessing attackers, does not stop this attacker.

Table 2 lists every field of the forged packet.

Field	Value
IP src (spoofed)	10.0.2.10 (the real server)
IP dst	10.0.1.10 (the victim client)
TCP sport	8000 (server's real listening port)
TCP dport	victim's ephemeral port (read from the sniffed packet)
TCP flags	RST only
TCP seq	the sniffed client ACK value (exact match)
Payload	none

Table 2: Fields of the forged RST packet.

The attacker fires on *every* sniffed client-to-server segment, rather than a single carefully timed shot: a late RST simply falls outside the client's already-advanced receive window and is silently discarded, with no side effects and no signal to the attacker. Since failure is this cheap, and a stream carries many packets, treating every sniffed segment as an independent, equally valid attempt compounds the overall probability of success across the life of the connection.

Both the packet-sniffing and packet-sending paths are implemented with raw `AF_PACKET` sockets and hand-rolled `struct`-based header parsing/construction, rather than a higher-level packet library. Section 5.3 explains why this specific implementation choice mattered for the attack's actual success rate.

4 Attack Process and Expected Outcome

The attack proceeds in the following steps:

1. The streaming server is started on h3 and begins listening.
2. The client (h1) requests the video and playback begins normally.
3. The attacker's visibility is enabled (the port mirror, Section 3.3), modeling the point at which an ARP-spoofing attacker would have already established a man-in-the-middle position.
4. The attacker (h2) sniffs h1's outgoing segments to h3 and, for each one, immediately forges and sends a RST to h1 using the field values in Table 2.
5. If a forged RST arrives while its sequence number still exactly matches h1's current expected value, h1's TCP stack resets the connection immediately.

Expected outcome. The client’s connection to the server should terminate abnormally – not via the ordinary graceful FIN/FIN,ACK exchange, but via an unexpected RST – part-way through the video, before the full file has been transferred. The video player should visibly freeze or report an error, and any subsequent playback attempt over the same (now-dead) connection should fail. Because the design depends on winning a race between the attacker’s short LAN path and the server’s longer, WAN-routed path, this outcome is expected *most of the time under realistic network conditions*, but is understood from the design stage onward to be probabilistic rather than absolutely guaranteed on every single attempt.

5 Observed Outcome

In short: yes, the attack was successful. Under every network condition realistic for actual video streaming (1–50 Mbit/s WAN bandwidth, Table 3), it interrupted playback essentially deterministically, in a single forged packet per trial. The rest of this section presents that outcome in detail: qualitative evidence from the attacker, victim, and server (Section 5.1), quantitative success-rate measurements across network conditions (Section 5.2), and an explicit account of the two specific cases where the observed outcome did not simply match the naive expectation, and why (Section 5.3).

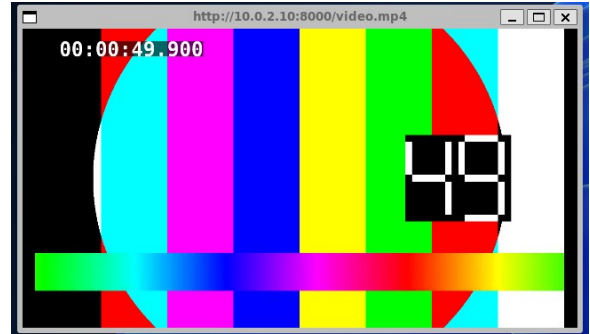
5.1 Qualitative Observations

```

root@LAPTOP-47N20LVP:/mnt/e/CSE_406/Project/streaming# python3
server.py
* Serving Flask app 'server'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production
deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8000
* Running on http://10.0.2.10:8000
Press CTRL+C to quit
[server] 10.0.1.10 GET /video.mp4 Range=bytes=0-
10.0.1.10 - - [19/Sep/2026 00:45:38] "GET /video.mp4 HTTP/1.1"
206 -

```

(a) Server (h3): incoming request, 206 response



(b) Client: normal playback, before the attack

Figure 2: Baseline, before the attack: the server logging the client’s request (left) and the client playing normally (right).

Attacker-side output. A representative successful run (Figure 3, left):

```

[attacker] resolved 10.0.1.10 -> 0a:eb:85:14:aa:87
[attacker] listening on h2-eth1 (raw socket, no Scapy dissection in the hot path)
[attacker] will forge RST packets as 10.0.2.10:8000 -> 10.0.1.10 via h2-eth0
[attack] #0001 00:51:03.464 fired RST -> 10.0.1.10:42532 seq=405514071
[attack] 10.0.1.10:42532 closed itself (flags=R) -- no longer targeting this flow
RESULT client_port=42532 attempts=1 time_to_close=0.0037872791290283203

```

A single forged RST killed the connection. The client’s own kernel then generated a RST of its own back toward the server, for the next already-in-flight real segment (arriving for a connection that no longer exists locally) – an independent, attacker-observable confirmation that the kill worked, without needing to look at the victim’s screen at all.

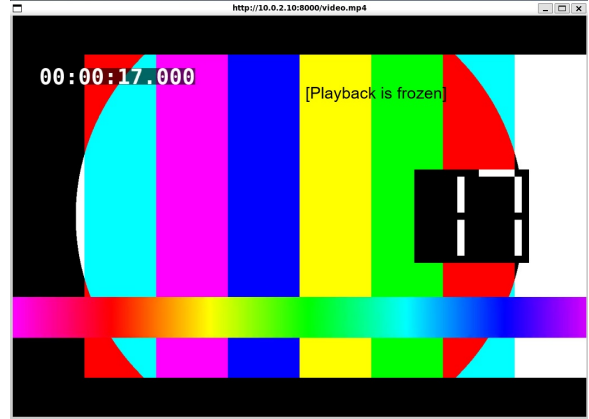
Victim-side output. The player’s own error output on a successful attack (Figure 4):

```

"Node: h2"
root@LAPTOP-47N20LVP:/mnt/e/CSE 406/Project/attack# python3 attacker.py
[attacker] resolved 10.0.1.10 -> 0a:eb:85:14:aa:87
[attacker] listening on h2-eth1 (raw socket, no Scapy dissection in the h
ot path)
[attacker] will forge RST packets as 10.0.2.10:8000 -> 10.0.1.10 via h2-e
th0
[attack] #0001 00:51:03.464 fired RST -> 10.0.1.10:42532 seq=405514071
[attack] 10.0.1.10:42532 closed itself (flags=R) -- no longer targeting t
his flow
RESULT client_port=42532 attempts=1 time_to_close=0.0037872791290283203

```

(a) Attacker (h2): fires one forged RST, connection closes immediately



(b) Client: playback frozen at the moment of the attack

Figure 3: The attack: a single forged RST (left, note `attempts=1`) kills the connection, and the client’s playback freezes at the same moment (right).

```

[mov,mp4,m4a,3gp,3g2,mj2] Packet corrupt (stream = 0, dts = 260096).
[mov,mp4,m4a,3gp,3g2,mj2] stream 1, offset 0xa5d51: partial file
[http] Stream ends prematurely at 678920, should be 3985444
[h264] Invalid NAL unit size (393 > 64).
[h264] Error splitting the input into NAL units.

```

Only 678,920 of the expected 3,985,444 bytes ($\approx 17\%$) were received. This is the player’s own confirmation that the underlying TCP transfer was severed abnormally, not merely slow – a slow-but-intact download would not produce a “partial file” / “ends prematurely” error at all. (When `curl` is used instead of a GUI player for scripted trials, the equivalent signal is a non-zero exit code, e.g. 56, “Recv failure: Connection reset by peer”.)

Server-side output. The server’s own log shows nothing unusual:

```

[server] 10.0.1.10 GET /video.mp4 Range=bytes=0-
10.0.1.10 - - [...] /Sep/2026 [...] "GET /video.mp4 HTTP/1.1" 206 -

```

The server keeps sending data into a connection that is already dead from the client’s perspective, until its own transport-layer error/timeout handling eventually notices. This asymmetry – the attacker targets the client, and the server has no direct indication anything is wrong – is a direct consequence of Attack A’s design (Section 3.4).

5.2 Quantitative Results

To avoid relying on a single anecdotal demo run, we built an automated experiment harness that drives the whole topology (server, attacker, client) through N repeated trials per condition, with success measured objectively – by comparing downloaded byte count against the video’s known full size, and by the client’s own exit code – rather than by eyeballing a player. Each trial gives the download a head start proportional to the estimated total transfer time before starting the attacker, modeling an attacker that gains on-path visibility partway through an already-flowing stream (as the ARP-spoofing precondition would produce in practice), rather than one present since before the TCP handshake.

Table 3 reports the attack’s success rate and mean attempts-to-kill across a range of WAN bandwidths (20 trials per condition, defense disabled).

```

"Node: h1"
ALSA lib conf.c:5182:(_snd_config_evaluate) function snd_func_card_in
um returned error: No such file or directory
ALSA lib confmisc.c:422:(snd_func_concat) error evaluating strings
ALSA lib conf.c:5182:(_snd_config_evaluate) function snd_func_concat
returned error: No such file or directory
ALSA lib confmisc.c:1334:(snd_func_refer) error evaluating name
ALSA lib conf.c:5182:(_snd_config_evaluate) function snd_func_refer r
eturned error: No such file or directory
ALSA lib conf.c:5705:(snd_config_expand) Evaluate error: No such file
or directory
ALSA lib pcm.c:2664:(snd_pcm_open_noupdate) Unknown PCM default
SDL_OpenAudio (1 channels, 44100 Hz): ALSA: Couldn't open audio devic
e: No such file or directory
No more combinations to try, audio open failed
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x7b19b0000cc0] Packet corrupt (stream = 0
, dts = 260096).
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x7b19b0000cc0] stream 1, offset 0xa5d51:
partial file
[http @ 0x7b19b0001a00] Stream ends prematurely at 678920, should be
3985444
[h264 @ 0x7b19b0253000] Invalid NAL unit size (393 > 64).0B f=0/0
[h264 @ 0x7b19b0253000] Error splitting the input into NAL units.
244.90 M-V: -0.030 fd= 2 aq= 0KB vq= 0KB sq= 0B f=0/0

```

Figure 4: Victim’s own terminal output (ffplay), confirming the underlying transfer was severed abnormally partway through the file.

WAN bandwidth	Est. segment gap	Success rate	Mean attempts
1 Mbit/s	≈24 ms	100%	1
5 Mbit/s	≈4.8 ms	100% (20/20)	1.0
12 Mbit/s	≈1.9 ms	100% (20/20)	1.05
20 Mbit/s	≈1.2 ms	100% (20/20)	1.05
50 Mbit/s	≈0.46 ms	100% (20/20)	1.0
100 Mbit/s	≈0.23 ms	0% (0/11)	—

Table 3: Attack success rate vs. WAN bandwidth, defense disabled.

We also captured packets directly on the victim to verify *why* the attack wins when it does: converting the forged RST’s raw sequence number back to the connection’s relative byte offset shows it lands exactly on the real stream’s current boundary every time (never a guess), and its arrival consistently precedes the real next segment by a small but measurable margin (observed margins: 5.5 ms, 10 ms, and 3.2 ms across three captured examples) – a direct, measured demonstration of the attacker’s short-LAN-path-vs-long-WAN-path advantage described in Section 3.

5.3 Expected vs. Observed Outcome, and Where They Differ

For the great majority of tested conditions (WAN bandwidths from 1 to 50 Mbit/s, which covers everything from a poor mobile connection to a high-quality stream), the observed outcome matched the expected outcome exactly: the attack succeeded, essentially deterministically, in a single shot per trial.

Two points are worth reporting explicitly because the observed outcome did *not* simply match expectations on the first attempt, and understanding why is itself informative.

Divergence 1: implementation latency, not network conditions, was the first real bottleneck. An initial version of the attacker used a well-known Python packet library (Scapy) for both sniffing and crafting packets. Under automated, repeated trials, this version showed a puzzling bimodal pattern: some connections died in exactly one shot, while others fired 600+ times across an entire streaming session and *never* landed a single hit – not the smoothly probabilistic behavior the design predicted. Packet-level diagnosis (a synchronized capture on the victim, comparing the timestamp of each forged RST against the real segment it was racing) found the actual cause: Scapy’s per-packet object-construction overhead was, on average, close to or above the real inter-segment gap on the shaped WAN link (≈ 24 ms at 1 Mbit/s). Once a single shot happened to be slow, a processing backlog began accumulating and never shrank back down – a textbook unstable queue – so the attacker ended up reacting to progressively staler packets for the rest of the connection’s life, with reaction latency measured climbing from 27 ms to over 16 *seconds* across one losing trial.

This was fixed by rewriting the attacker to use raw `AF_PACKET` sockets with hand-rolled header parsing/construction (avoiding Scapy’s per-packet overhead), and by always processing the newest queued packet rather than the oldest (bounding the worst case to one stale cycle instead of an ever-growing backlog). After this fix, measured reaction latency dropped to 0.1–10 ms (typically sub-millisecond), and the attack’s behavior across repeated trials became the near-deterministic one-shot success reported in Table 3. In short: the initial divergence between expected and observed behavior was not a flaw in the attack’s design, but an artifact of Python/Scapy’s processing overhead exceeding the available race margin – once that overhead was removed, observed behavior matched the design’s prediction.

Divergence 2: the race flips at unrealistically high bandwidth. At 100 Mbit/s, the observed success rate dropped to 0% (0/11 trials) – the opposite of the near-100% outcome at every lower bandwidth. This is explained directly by the same race-margin model: at 100 Mbit/s the real segment gap shrinks to roughly 0.23 ms, which is now comparable to or below the attacker’s own (already-optimized) processing latency. The attacker’s structural, topological advantage (one LAN hop vs. a router-mediated WAN path) is necessary but not sufficient once the legitimate traffic itself is moving faster than the attacker’s software can plausibly react. This is not a contradiction of the attack’s design – the design was explicit from the start that success depends on winning a race, not on a guaranteed exploit – but it is a concrete, measured boundary of where that race is actually won or lost, and 100 Mbit/s is well outside the range of realistic video streaming bitrates, so it does not undermine the attack’s practical effectiveness.

6 Defense Mechanism

6.1 Design

Rejected approach: router-side ingress filtering. An initial idea was to drop, at the router, any packet claiming to be from the server but arriving on the router’s LAN-facing interface – a genuine server packet could only arrive via the WAN side. This fails *topologically*, not merely as weak policy: the forged RST travels attacker $\rightarrow$ switch $\rightarrow$ client entirely within the LAN, since its destination (the client) is on the local subnet. It never reaches the router’s LAN interface at all, so a router-side rule is not on this attack’s path and cannot see the malicious packet.

Chosen approach: switch-side IP Source Guard. Every attacker-to-victim frame must physically pass through the shared LAN switch, which makes it the correct choke point. Real switches implement IP Source Guard by binding each port to the IP/MAC pair learned via DHCP snooping, then dropping any frame on that port whose source IP does not match the

binding. Since the attacker’s real IP is known by construction in our emulated topology, we install the equivalent access-control rule directly on the Open vSwitch switch:

```
ovs-ofctl add-flow s1 "priority=100,in_port=<attacker's port>,
ip,nw_src=10.0.2.10,actions=drop"
```

This rule drops only IPv4 packets that forge the server’s address while entering on the attacker’s specific switch port; the attacker’s own real traffic (using its own IP) and all ARP traffic are unaffected, and the rule does not interfere with the mirror-based visibility mechanism from Section 3.3 – the defense stops forged *delivery* of the attack packet, not the (separately assumed) ARP-spoofing visibility precondition itself.

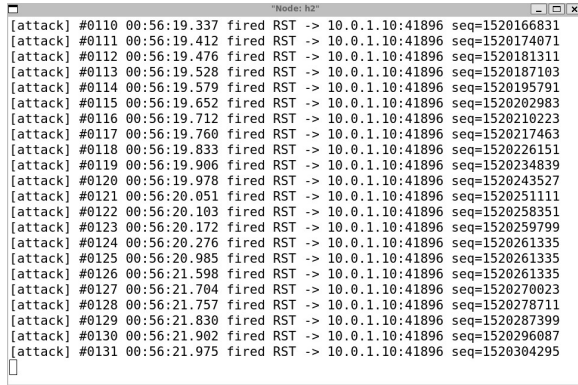
6.2 Defense Evaluation: Does It Work as Expected?

The expectation was that this defense should reduce the attack’s success rate to 0%, regardless of network bandwidth, since it operates as a deterministic access-control check at the switch rather than depending on any timing race. Table 4 reports the measured before/after comparison across the same repeated-trial methodology used in Section 5.2.

WAN bandwidth	Defense off (success rate)	Defense on (success rate)
5 Mbit/s	100%	0%
12 Mbit/s	100%	0%
20 Mbit/s	100%	0%
50 Mbit/s	100%	0%

Table 4: Attack success rate with the defense disabled vs. enabled, 20 trials per condition.

The observed outcome matched the expectation exactly: with the defense enabled, every trial across every tested bandwidth resulted in a full, uninterrupted download – the attacker fired hundreds of forged RSTs per trial (since it has no awareness of the defense and simply keeps trying) and none of them reached the victim.

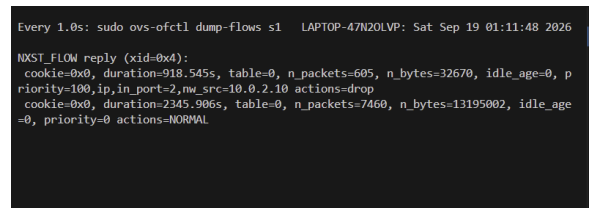


```

[attack] #0110 00:56:19.337 fired RST -> 10.0.1.10:41896 seq=1520166831
[attack] #0111 00:56:19.412 fired RST -> 10.0.1.10:41896 seq=1520174071
[attack] #0112 00:56:19.476 fired RST -> 10.0.1.10:41896 seq=1520181311
[attack] #0113 00:56:19.528 fired RST -> 10.0.1.10:41896 seq=1520187103
[attack] #0114 00:56:19.579 fired RST -> 10.0.1.10:41896 seq=1520195791
[attack] #0115 00:56:19.652 fired RST -> 10.0.1.10:41896 seq=1520202983
[attack] #0116 00:56:19.712 fired RST -> 10.0.1.10:41896 seq=1520210223
[attack] #0117 00:56:19.760 fired RST -> 10.0.1.10:41896 seq=1520217463
[attack] #0118 00:56:19.833 fired RST -> 10.0.1.10:41896 seq=1520226151
[attack] #0119 00:56:19.906 fired RST -> 10.0.1.10:41896 seq=1520234839
[attack] #0120 00:56:19.978 fired RST -> 10.0.1.10:41896 seq=1520243527
[attack] #0121 00:56:20.051 fired RST -> 10.0.1.10:41896 seq=1520251111
[attack] #0122 00:56:20.103 fired RST -> 10.0.1.10:41896 seq=1520258351
[attack] #0123 00:56:20.172 fired RST -> 10.0.1.10:41896 seq=1520259799
[attack] #0124 00:56:20.276 fired RST -> 10.0.1.10:41896 seq=1520261335
[attack] #0125 00:56:20.985 fired RST -> 10.0.1.10:41896 seq=1520261335
[attack] #0126 00:56:21.598 fired RST -> 10.0.1.10:41896 seq=1520261335
[attack] #0127 00:56:21.704 fired RST -> 10.0.1.10:41896 seq=1520270023
[attack] #0128 00:56:21.757 fired RST -> 10.0.1.10:41896 seq=1520278711
[attack] #0129 00:56:21.830 fired RST -> 10.0.1.10:41896 seq=1520287399
[attack] #0130 00:56:21.902 fired RST -> 10.0.1.10:41896 seq=1520296087
[attack] #0131 00:56:21.975 fired RST -> 10.0.1.10:41896 seq=1520304295

```

(a) Attacker’s terminal: 130+ forged RSTs fired, none succeed



```

Every 1.0s: sudo ovs-ofctl dump-flows s1 LAPTOP-47N20LVP: Sat Sep 19 01:11:48 2026
NXST_FLOW reply (xid=0x4):
 cookie=0x0, duration=918.545s, table=0, n_packets=605, n_bytes=32670, idle_age=0, p
 riority=100,ip,in_port=2,nw_src=10.0.2.10 actions=drop
 cookie=0x0, duration=2345.906s, table=0, n_packets=7460, n_bytes=13195002, idle_age
 =0, priority=0 actions=NORMAL

```

(b) Switch’s own flow table: the drop rule’s `n_packets` counter

Figure 5: Defense evidence: the attacker’s own terminal (left) and the switch’s flow-rule counters (right, `ovs-ofctl dump-flows s1`, refreshed live via `watch`).

Figure 5 (left) shows the attacker’s terminal well past its 100th attempt against a single connection, still firing and still failing – exactly the behavior expected of a switch-level drop rather than a timing loss (a timing loss would still occasionally succeed; a hard drop never does). We confirmed the mechanism directly, not just its effect, by inspecting the switch’s own flow-rule

counters (Figure 5, right): the installed rule (`priority=100,ip,in_port=2,nw_src=10.0.2.10 actions=drop`) shows `n_packets=605` and climbing under `watch`, in step with the attacker’s own attempt count – direct confirmation that the forged packets are being matched and dropped at the switch, not failing to arrive for some unrelated reason. Unlike the attack itself (Section 5.3), the defense’s effectiveness does not depend on network speed at all, since it is a deterministic access-control check rather than a timing race – this is exactly the behavior expected of the design.

7 Discussion, Scope, and Limitations

The threat model and assumptions stated in Section 2 bound the scope of what was implemented and evaluated. In particular: ARP spoofing (the on-path visibility precondition) was assumed already satisfied rather than implemented, since it is a separately assigned project tool; only Attack A (forged RST to the client) was implemented and evaluated, not the converse Attack B; and the attacker deliberately never suppresses or delays legitimate traffic, only observes and injects.

Within that scope, some further limitations are worth noting. The race behavior in Section 5.2 was characterized only for the specific topology and bandwidth/delay range tested; real-world WAN paths have more variable delay and jitter than our shaped emulation, which could shift the exact bandwidth at which the race flips (Divergence 2, Section 5.3) in either direction. RFC 5961’s challenge-ACK behavior for an *inexact* (guessed, rather than sniffed) sequence number was discussed conceptually during design but not separately implemented or measured, since our attacker always uses the exact sniffed value and so never exercises that code path. Possible extensions include implementing Attack B, an off-path sequence-guessing variant (to concretely exercise the challenge-ACK path RFC 5961 provides), and TTL-based anomaly detection as an additional, complementary defense layer.

8 Conclusion

We implemented, from scratch, a working TCP reset attack that reliably interrupts video streaming under realistic network conditions, using only self-crafted raw-socket code for both sniffing and packet construction. We measured its actual success rate and behavior across a range of network conditions rather than relying on a single demo run, and explicitly identified and explained the two cases where the observed outcome diverged from the naive expectation: an initial implementation-latency bottleneck (fixed, and confirmed fixed by re-measurement) and a genuine, expected-in-principle race-margin boundary at unrealistically high bandwidth. We further designed, implemented, and evaluated a switch-side IP Source Guard countermeasure that reduces the attack’s success rate from 100% to 0%, and confirmed that this defense behaves exactly as expected – deterministically, independent of network speed – across every tested condition.

Contribution split.

Member	Contribution
2105094: Omar Abdur Razzaque	[FILL IN: describe this member’s actual contribution]
2105104: Rubaiyat-E-Zaman	[FILL IN: describe this member’s actual contribution]

References

1. RFC 793 — Transmission Control Protocol.
2. RFC 5961 — Improving TCP's Robustness to Blind In-Window Attacks.
3. Ishtiaq, Faruk, Hossain — *TCP Reset Attack on Video Streaming*, NSysS 2019 poster (BUET).
4. Watson, P. — *Slipping in the Window: TCP Reset Attacks*.
5. [FILL IN: any additional references you used]